Bit Manipulation & Bitmasking Cheat Sheet (C-centric)

1. Bitwise Basics

Operation	C Code	Description
Set bit `i`	`x |= (1 << i);`	Sets bit `i` to 1
Clear bit `i`	`x &= ~(1 << i);`	Sets bit `i` to 0
Toggle bit `i`	`x ^= (1 << i);`	Flips bit `i`
Check bit `i`	`x & (1 << i)`	Is bit `i` set? (nonzero if yes)
Extract bit `i`	`(x >> i) & 1`	Returns 0 or 1

2. Counting & Manipulating Bits

Trick	C Code	Purpose
Count set bits (Kernighan)	`while (x) { count++; x &= (x - 1); }`	O(1s in x)
Clear lowest set bit	`x & (x - 1)`	Removes rightmost 1
Isolate lowest set bit	`x & -x`	Keeps only rightmost 1
Is x power of 2?	`x > 0 && (x & (x - 1)) == 0`	True only for powers of 2
XOR all nums to find single	`res ^= x;` in loop	Only unique survives

3. Bitmask Patterns

Pattern	C Code	Use Case
Subset loop over `n` bits	`for (int mask = 0; mask < (1 << n); mask++)`	Enumerate all subsets
Check if bit `i` is in mask	`if (mask & (1 << i))`	Include element `i` in subset
Count bits in a mask	Use Kernighan's or `__builtin_popcount(mask)` (GCC)	Count elements in subset

4. XOR Logic Patterns

Pattern	Result
`x ^ x`	0
`x ^ 0`	x
`a ^ b ^ b`	a
Used to swap:	`a ^= b; b ^= a; a ^= b;`

5. Advanced Tricks

Trick	C Code	Description
Reverse bits	Use loop: shift + mask	For FFT, binary mirror
Generate Gray Code	`gray = n ^ (n >> 1)`	Used in hardware encoders
Convert Gray -> Binary	`while (shift) binary ^= (gray >> shift--)`	Recursive XOR fold
Bit parity (even/odd 1s)	XOR all bits or use `__builtin_parity()`	HW parity check
Round up to power of 2	`x--; x |= x >> 1; x |= x >> 2; ... x++;`	Fills with 1s then adds 1